

MOOD BOARD

金融資料探勘

作業名稱：運用 AI 分析看漲投資人及看跌投資人每日隱含 波動率的平均與樣本標準差，
並將臺指 選擇權買賣權比（PUT/CALL RATIO, PCR）納入整合分析，並完成 PROMPT
實作與優化紀錄

姓名：林欣慈

學號：411336045

組別：2

負責年度：2022 年

繳交日期：2026 年 4 月 30日

作業目的

本作業旨在透過 Python 程式自動化處理大規模的台指選擇權 (TXO) 逐筆交易資料。目標是從雜亂的原始 CSV 檔中，篩選出符合流動性要求的交易樣本，並結合標的指數價格與無風險利率，建立一個包含隱含波動率 (IV) 計算基礎的「最終索引檔」，以便後續進行市場情緒與 PCR (Put-Call Ratio) 之分析。

原始資料說明

- 資料來源：期交所逐筆交易資料 (Daily Quotes)。
- 原始欄位：包含交易日期、商品代碼、履約價、買賣權別、成交價格、成交量等。
- 參考檔案：參考「Path_教學_0305.csv」定義基礎索引欄位結構，並參考「Path_教學_修正 Rf 版.xlsx」進行最終格式微調。

資料建構流程

- 日期整理與欄位改名：將原始資料中不一致的日期格式統一為 YYYY-MM-DD，並將欄位名稱標準化（如：Strike Price 改為 K）。
- 建立 File 索引：掃描資料夾內所有 CSV 檔案，建立與交易日期對應的檔案路徑索引。
- Contract 與 ContractExpiryDate：根據商品代號標記 TXO，並與 taifex.csv 串接，找出該筆交易對應的到期日。
- Maturity (T)：計算交易日至到期日的天數，並除以 365 轉換為年化時間。
- Rf 合併：根據交易日期，從利率表中 VLOOKUP (或 Merge) 出當日的無風險利率。

最終索引檔欄位說明

欄位名稱	資料型態	意義與用途
Date	Datetime	交易日期，作為合併標的價格與利率的主鍵。
S0	Float	當日標的指數 (台指期或大盤) 收盤價。
K	Float	選擇權履約價。
Type	String	買權 (Call) 或 賣權 (Put)。
T	Float	到期時間 (Maturity)，年化後用於 BS 模型。
Rf	Float	無風險利率，IV 計算之必要參數。
Volume	Int	成交量，用於篩選高流動性樣本及計算 PCR。

成果展示

1	Date	File	S0	Maturity	Contract	rf
2	2022/1/3	OptionsDaily_2	18270.50977	16	202201	0.00755
3	2022/1/4	OptionsDaily_2	18526.34961	15	202201	0.00755
4	2022/1/5	OptionsDaily_2	18499.96094	14	202201	0.00755
5	2022/1/6	OptionsDaily_2	18367.91992	13	202201	0.00755
6	2022/1/7	OptionsDaily_2	18169.75977	12	202201	0.00755
7	2022/1/10	OptionsDaily_2	18239.38086	9	202201	0.00755
8	2022/1/11	OptionsDaily_2	18288.21094	8	202201	0.00755
9	2022/1/12	OptionsDaily_2	18375.40039	7	202201	0.00755
10	2022/1/13	OptionsDaily_2	18436.92969	6	202201	0.00755
11	2022/1/14	OptionsDaily_2	18403.33008	5	202201	0.00755

問題與修正

- 日期格式問題： 原始資料日期格式混雜，透過 `pd.to_datetime` 並指定 `errors='coerce'` 修正非標準字串。
- 契約判定與過濾： 初始分析包含非 TXO 商品，修正為僅保留 `Product == 'TXO'` 且成交量大於 30 口之樣本。
- Rf 合併異常： 部分假日無利率資料，採用 `fillna(method='ffill')` 以最近交易日利率遞補。
- 到期日對應： 選擇權週到期與月到期混淆，修正邏輯以精準匹配每週三之結算日。

AI Prompt 實作與 優化紀錄摘要

- 任務： 撰寫向量化 GPU 加速程式碼以計算 IV。
- 初始 Prompt： 「如何用 Python 算選擇權隱含波動率？」
- 修正重點： 指定使用 Bisection Method (二分法)、CuPy 庫，並要求處理大規模 DataFrame 的向量化架構。
- 最佳 Prompt： 「請使用 Python 與 CuPy 撰寫一個向量化函數，利用二分法在 GPU 上求解 Black-Scholes 隱含波動率，並自動處理 $T \leq 0$ 的異常值。」
- 協助成果： 運算速度提昇約 50 倍，成功在 5 分鐘內處理完三年的逐筆資料。

學習反思

AI 在本次作業中極大地縮短了調試 (Debug) 複雜數學公式的時間，特別是在處理 Black-Scholes 向量化運算時。然而，AI 的限制在於對「台灣期交所特定資料格式」的理解有限，仍需人工介入調整日期對應邏輯。最重要的修正點是在 Prompt 中加入具體的演算法名稱與性能需求。未來改進方向將嘗試讓 AI 協助建立更自動化的資料品質檢查 (Data Quality Check) 流程。